



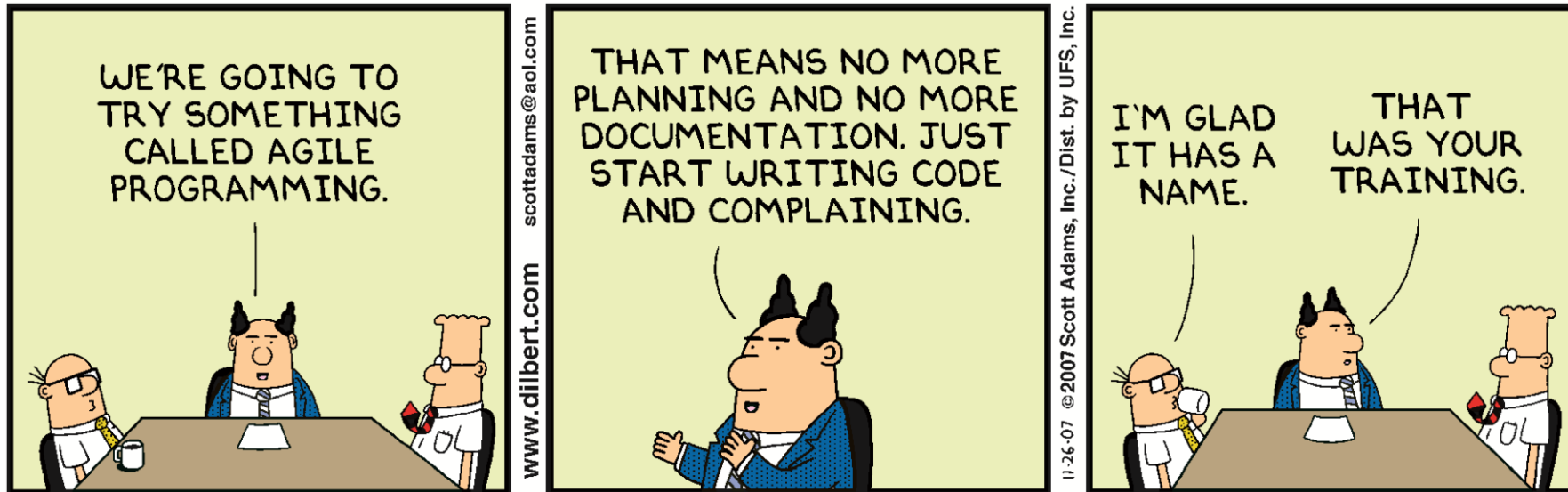
Carnegie Mellon University
Master of
Software Engineering

Architecture and Agile, technical debt

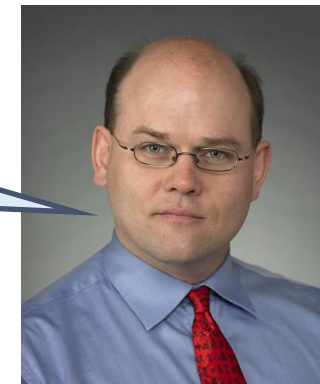
LG Architecture Training Program

Paulo Merson

A myth from way back...



In reality, agile teams are more likely to model than traditional teams



Scott Ambler

Architecture in agile teams

- Agile developers can design
- Agile architects can code
- Stop designing as soon as you're ready to code, not a moment later than that
 - From then on, design a little, code a little, design a little, code a little, ...

If you think good architecture is expensive, try bad architecture.



Brian Foote



Say *no* to BDUF and *yes* to IDUF

- The goal for agile architecture is not to avoid design, it's to avoid “big design up front” (BDUF)
 - Ample scope design decisions are made early (“important design up front”)
 - In each sprint, decisions are expanded and detailed to implement the sprint features

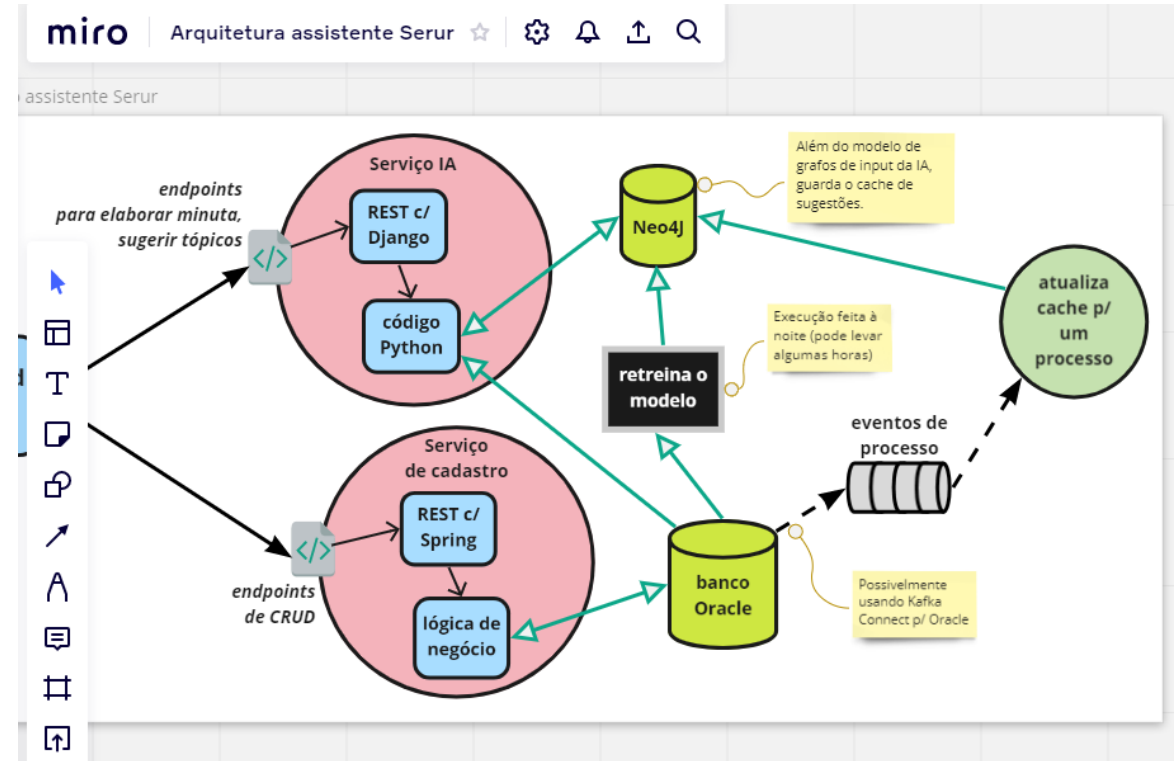
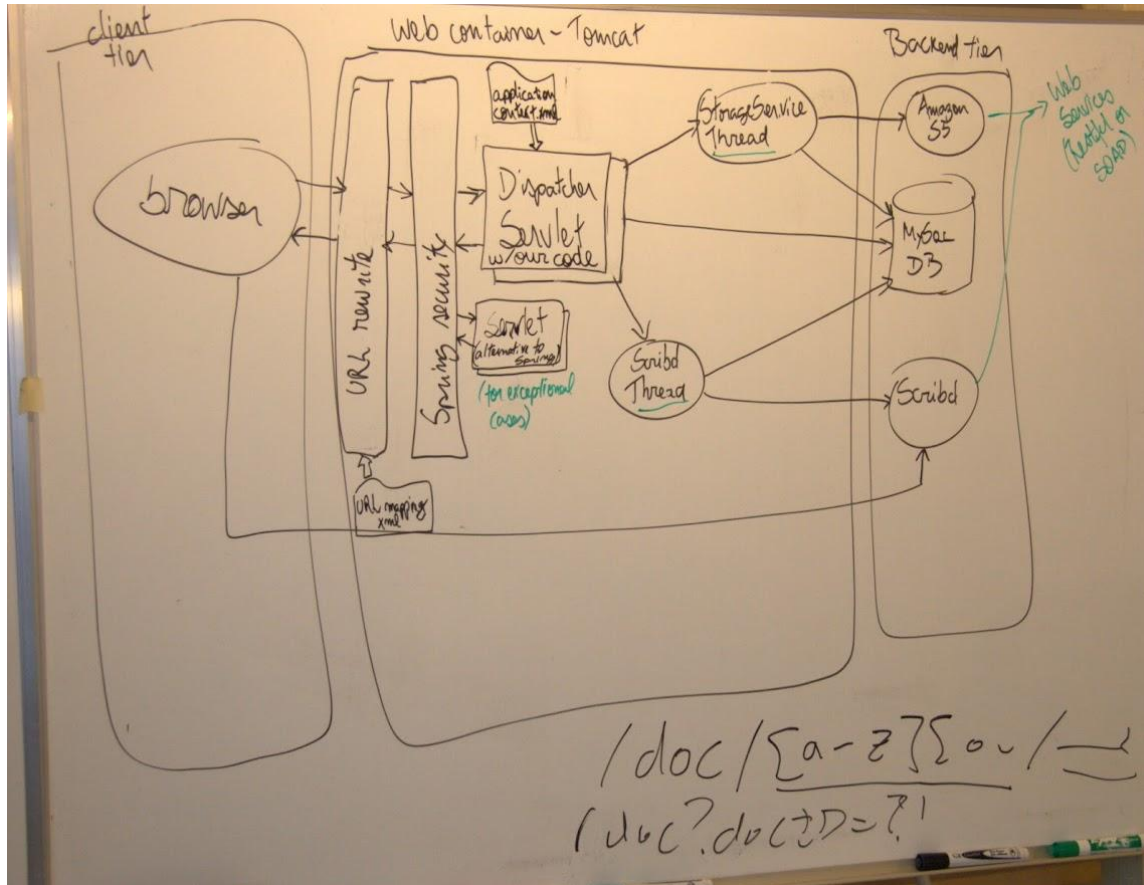
*Big design up front is dumb
but doing no design up front
is even dumber.*



Dave Thomas



Sketches on a whiteboard can be your architecture views



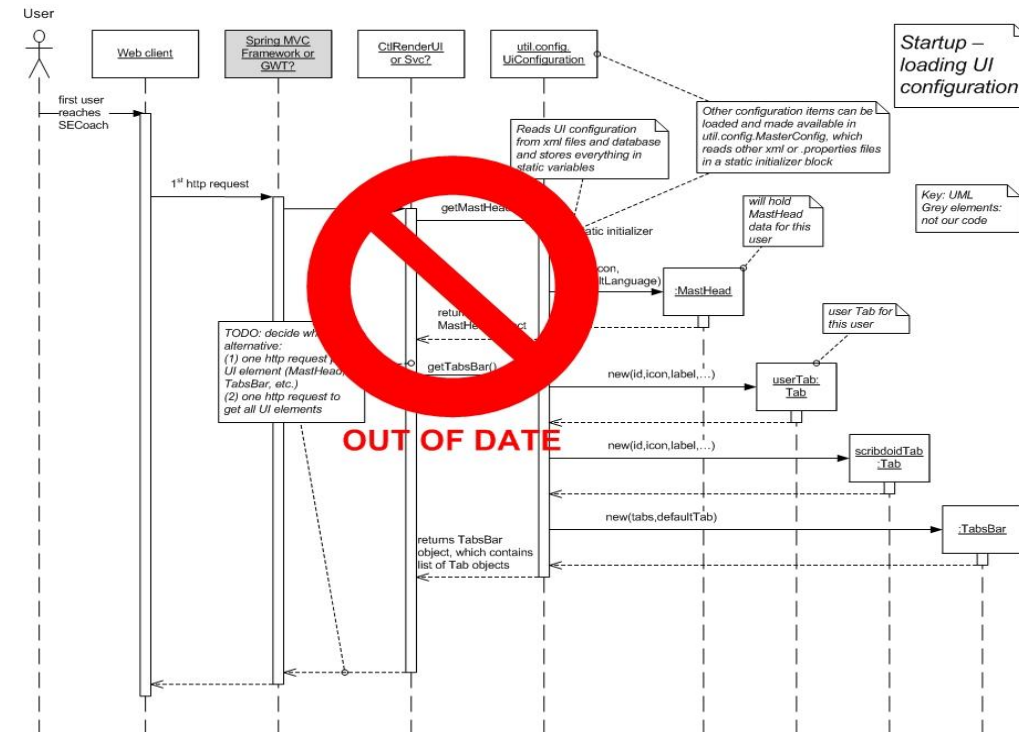
For online whiteboarding use tools like miro, mural, figma, lucidspark, excalidraw

What to do with out-of-date diagrams?

- Diagrams (and any accompanying prose) are created to aid the design discussion
 - Once the code is written, the diagram has served its main purpose
 - It's common for code sooner or later to differ from the original design

What to do with a design diagram that no longer reflects what's in the code?

- *Update it: ideal but not always feasible*
- *Leave it as is: worst option*
- *Erase or mark as out-of-date: OK option!*



Deliver Fast with Confidence

Joe Yoder

Agile Myths

- Simple solutions are always best
- We can easily adapt to changing requirements (new requirements)
- Scrum/TDD will ensure good Design/Architecture
- Good architecture simply emerges from “good” development practice
- You always go fast when doing agile
- Make significant architecture changes at the last moment

“www.agilemyths.com”

MYTHBUSTERS



7

Technical debt

- Metaphor introduced by Ward Cunningham to focus on bad code [Cunningham92]
 - Since extended to bad architecture and beyond (bad tests, requirements, documentation, technology)
- It is visible to developers, invisible to end users

Accumulating some debt to speed up time to market might be a good thing



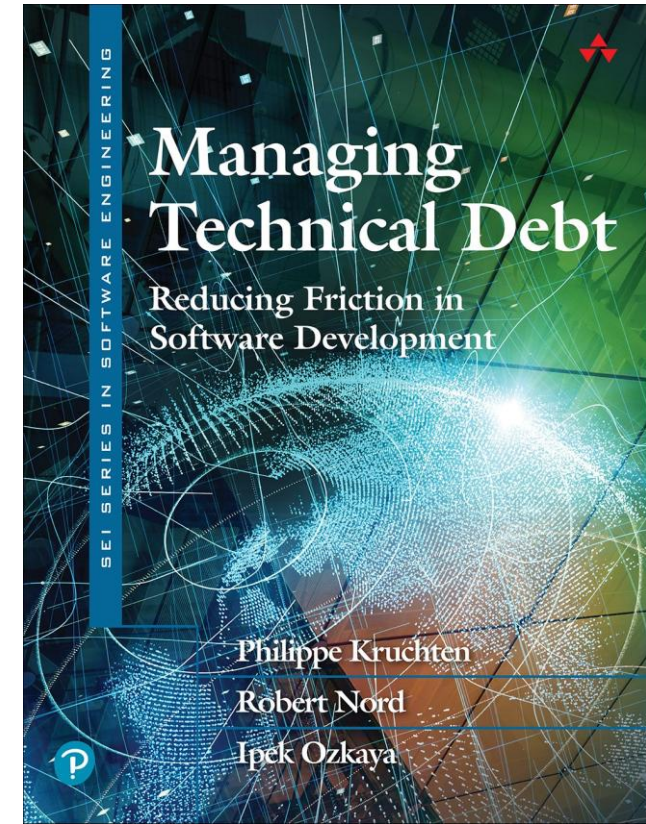
Ward Cunningham

How to address technical debt

- Identify and record technical debt
 - Static code analysis tools can spot problems and even calculate debt
 - Record debt-related tasks in the backlog (identified as technical debt)
- Don't let it grow continuously
- Pay incrementally
 - Establish periodic sprints to address technical debt tasks



*If feels good to
pay off debt*



What causes technical debt

- Schedule pressure
- Lack of technical knowledge
- Poor (or inexistent) development process
- Lack of quality verification
- Basic incompetence
- Poor architecture understanding



*Designing an architecture without documenting it
is like winking at a girl in the dark. You know what
you're doing, but nobody else does.*
[Clements10]





Questions? now or later

Paulo Merson
pmerson@cmu.edu